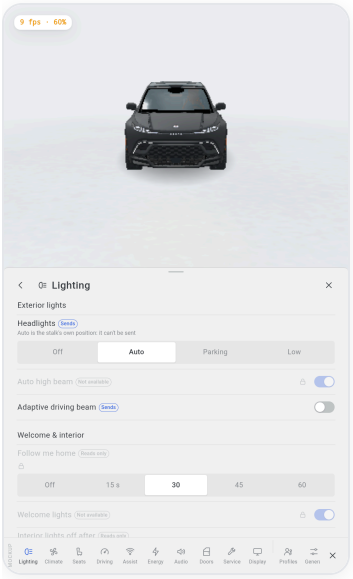
FISKER OCEAN · PULSE HUD

Make the Pulse HUD feel finished

An evidence-based polish layer on top of the chosen layout: feedback that confirms from the car not the tap, a control catalog that stops greying everything alike, and automotive-safe motion, night and empty-state behaviour — with a phased development plan.



Date 2 Nov 2026 **Branch** pulse-coexist **Builds on** Pulse HUD UX options **Scope** interaction detail, page-only unless noted

The Pulse HUD already behaves correctly; what makes it read as unfinished is a thin layer of interaction detail the evidence shows how to fix cheaply. Three moves carry most of the value. First, feedback must be driven by the car, not the tap: acknowledge a touch within **~100 ms** (ideally ~50 ms) with a cheap visual press-state decoupled from the 3D frame (USPTO 10310604; NN/g, Response Times), then settle to a "confirmed" resting state only when the car's own frame comes back — and for the control trial's hard case, where the head unit re-sends its own value every cycle, treat the send as optimistic and **visibly snap the control back to the car's value with a brief inline note** rather than letting a dead optimistic value sit (DEV Community, optimistic UIs). Second, the catalog's greyed rows must stop looking alike: the literature splits **read-only** (a live value the driver should read — keep it full-contrast, 4.5:1, reachable) from **disabled** (muted, lower-contrast, with a discoverable reason) from **temporarily-unavailable** (parked-only / source-disconnected — same muted base plus a lock glyph and "available when parked" copy), and the current HUD collapses all three into one faint grey (Carbon, Read-only states; UX Tigers, Inactive Buttons). Third, polish must respect the driving boundary: still non-driving motion while moving, hold a calibrated night brightness floor so the screen never becomes a glare source, and present empty/offline slots calmly rather than with alarm chrome (Google, Visual principles; engineered.at, instrument clusters). The single cheapest high-impact win sits outside all three: the FPS/debug chip ships `showFps: true` in `main.js` and overlays every screen — turning it off by default is a one-line change. This report is the interaction-detail layer on top of whichever layout the earlier options report recommends (Glance Strip, Tile Dash, Map Centre, Ocean Native); it assumes that layout work, not the overlay set it replaces.

Acknowledge the tap in 100 ms, confirm from the car, not from the touch

The response-time budget the whole design rests on is NN/g's three limits, retrieved directly and reliable: **0.1 s** feels instantaneous and is where a selection highlight must appear for direct manipulation; **1.0 s** keeps a train of thought but needs a "working" cue; **10 s** is the attention limit and needs determinate progress (NN/g, Response Times). A CAN send plus the car's reply almost always resolves well under a second, so the HUD lives almost entirely in the 0.1 s tier: the dominant need is an instant press acknowledgement plus a confirmed-or-reverted resolution, and the 1 s "working" and 10 s "percent-done" tiers rarely apply. In a driving context the acknowledgement target tightens — human-factors and patent literature converge on **"immediate" ≈ under 50 ms with ~100 ms a common ceiling**, and research puts the feels-instant window at **20–50 ms** (USPTO 10585481; USPTO 10310604; Springer, touch response time). Because the Pulse page shares a frame budget with a heavy three.js scene (the worker/main-thread split the repo is built around, and the parked throttle that drops the scene to ~20 Hz), the press acknowledgement must be a cheap, immediate visual state change — a press/fill on the control itself — that never waits on the 3D frame; where the Android app can, a short haptic or audio cue lets the driver confirm "without a long glance," which keeps the confirmation inside the ~2 s glance budget (Immersion, haptic latency; Google, Visual principles).

The resting visual language should borrow Material 3's state-layer overlays — **hover 8%, focus 10%, pressed 10%, dragged 16%** role-color over the component — which come from the androidx Compose `StateTokens` source rather than a live read of m3.material.io, and which an older material-web table (12% focus/pressed) superseded (androidx Compose StateTokens via port; material_3_expressive, Dart). Motion durations should map to M3's 16-step scale — short tiers **50/100/150/200 ms** for press and state-layer changes, medium **250–400 ms** for card and panel transitions, long **450–600 ms** reserved for deliberate changes — with the standard easing `cubic-bezier(0.2, 0.0, 0.0, 1.0)` and ripple-style press feedback near a **~75 ms fade-in / ~225–280 ms**

settle; these rest on the Flutter M3 generator and component ports, not a direct spec read, so treat them as the token values rather than a verified clause (hydrolysis-m3 motion tokens; hydrolysis-m3 ripple timings).

The decisive architectural point is that the "confirmed" state must be driven by the car's reply frame, not by the tap. This is the wait-for-confirmation model: the canonical value comes back on the bus, so the control should show a brief "pending/sent" state the instant the finger lands (satisfying the 0.1 s rule), then settle to "confirmed" when the car's next frame matches — a short success dwell of **~1.5 s** before the resting state is a reasonable borrowed convention (Carbon's inline-loading success dwell, version-specific to v10) (Carbon v10, inline loading). This matters because DESIGN.md already specifies a ~1.5 s pending glyph fade, and the earlier options report found there is **no `pending` class in the code** — the state is designed but unbuilt. Failure and error should surface inline near the control, not only as a transient toast, because a snackbar auto-dismisses in a **4–10 s** window and is easy to miss while driving; the persistent inline control state (the control visibly showing the car's current value) is the reliable source of truth, with the snackbar supplementary and reserved for brief, low-stakes confirmations or reversible actions offering Undo (the longer **8–10 s** end of the band) (Material snackbar, mirrored; Firefox Acorn, Snackbar; Carbon, loading usage). No major design system publishes a named "your value was accepted then overridden" state, so that case must be composed from the failure, transient-highlight, and inline-state patterns — which is the next section.

The head unit's override needs a visible snap-back, not silence

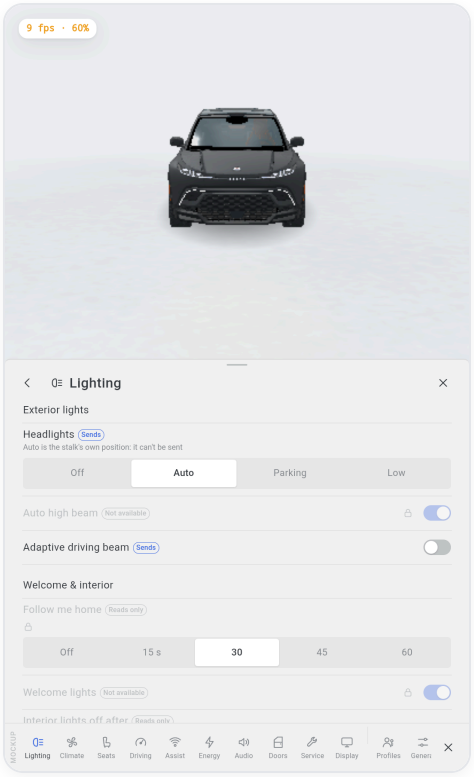
The control trial (Settings → Debug → Unlock controls, parked only, off by default) is the one place the HUD can send a head-unit-owned frame, and `e2e.js` does it honestly — it copies the head unit's latest frame, changes one field, continues the alive counter and fixes the CRC-8 J1850 checksum — but the head unit re-sends its own cyclic frame every cycle, so a user's change can be silently overwritten. This maps cleanly onto optimistic UI's "reconcile to canonical data": the car's cyclic frame **is** the canonical value, so the HUD should treat its own send as optimistic, apply it instantly, and reconcile against the car's next frames (DEV Community, optimistic UIs; openreplay, optimistic updates). The non-negotiable rule from the optimistic-UI literature is that automatic rollback is **not enough on its own — the UI must always tell the user about the revert**; React's `useOptimistic` reverting on a failed transition still needs an explicit message (DEV Community, useOptimistic). Carry forward the source caveat here: optimistic UI is widely practiced **practitioner convention, not research-validated** — there is no formal study behind it, only perceived-performance reasoning (skills.cat, feedback-design, secondary) — and no automotive source prescribes exactly how to animate an override on a vehicle HUD, so the treatment below is assembled from web-app optimistic practice plus the transient-highlight and inline-state conventions.

The concrete treatment: on tap, snap the control to the requested value immediately (0.1 s rule) and show a short **"sent"** pending state. Because `cancmd.js` sends an event message three times 20 ms apart and holds a cyclic one for three 100 ms cycles before the idle frame, the reconciliation window should be a small multiple of the cyclic cadence — open a bounded window of roughly **1.5–2.5 s** (tune to the frame cadence; no source fixes this number) and watch the car's incoming frames. If a confirming frame arrives, settle to "confirmed" with the ~1.5 s success dwell. If the car's next frames instead keep asserting the old value, resolve to an explicit **"overridden"** state: flash the control's value back to the car's value as a transient highlight (a ~200–280 ms state-layer settle so the eye catches the change) and show a brief inline note on the row — templated, e.g. *"The head unit changed this back."* If no confirming frame arrives within the window at all, fall to a **"didn't stick"** state rather than leaving the optimistic value showing. Keep the inline note as the persistent truth and, only if the user likely wants to retry,

add a supplementary snackbar (“Overridden by car — Retry”) at the longer 8–10 s end because it offers an action; never use a blocking dialog for a routine silent revert (Firefox Acorn, Snackbar; Mirakl, Snackbar, secondary). This is entirely page-side: the HUD already receives the car’s frames the reconciliation reads, so no Android or bridge change is needed.

Read-only, disabled, and parked-only must stop looking alike

The strongest finding in the unavailable-controls evidence is a distinction the HUD currently blurs. **Read-only** means a real value the user should read but cannot change — Carbon keeps it in the tab order, readable by screen readers, and its text **must still meet 4.5:1 contrast**; **disabled** means an action temporarily unavailable pending user action — it is skipped by screen readers and is allowed to drop below contrast (Carbon, Read-only states). Cloudscape draws the same line: disabled when interacting could cause an error because a prerequisite is unmet, read-only when the user must see a value but not change it (Cloudscape, disabled and read-only). The dominant usability failure of a bare greyed control is that it is a “communication dead end” — users cannot tell *why* it is off or *whether* it can be enabled — and disabled buttons “appear clickable but provide no response,” confusing users and shutting out keyboard and screen-reader users entirely (NN/g, Why Disabled Buttons Hurt UX; Smashing, disabled buttons). The project’s grey-not-hide rule is well-supported here: Nielsen’s line is to hide only what a user can *never* reach, but almost every greyed Pulse row is conditionally available (parked-only, trial-gated, source-dependent), so the rule is sound and the fix is reasons plus de-emphasis, not hiding — and this choice has automotive precedent, where Volvo, Honda, and Suzuki all grey-out rather than hide functions that cannot be selected, several of them gated on the vehicle being stopped (UX Tigers, Inactive Buttons; Volvo S80 support).



Today: the Lighting sheet greys a **read-only** value (Follow me home) and a **blocked** control (Welcome lights) to the same faint tone — the clutter this report resolves into three distinct states.

The HUD's three catalog categories map directly onto this literature, so the recommended three-way visual language is concrete. **Read-only / informational** (the catalog's "Reads only", every `live` value the car reports — `OWNED` and `NO_MSG`-but-readable rows): full-contrast value text at 4.5:1, normal row layout, only the edit affordance removed, an arrow (not disabled) cursor — it must read as live data the driver trusts, and Carbon's rule is to switch only *signifier* icons to a disabled token while keeping the value and context icons at full strength (Carbon, Read-only states; Design Systems Collective, read-only fields). **Disabled / blocked** (the `MOCK` and genuinely-unsupported rows): muted — ideally a desaturated version of the control's own accent rather than flat grey — with a lower-contrast but still-legible label, static on tap, and the reason hint; NN/g notes the label may be lower contrast than enabled but must stay legible and must not look clickable (NN/g, Button States). **Temporarily-unavailable** (parked-only, source-disconnected, head-unit-overwritten during the trial): the same muted base as disabled, distinguished by the reason wording and a **state glyph — a lock for parked-only** — because the user's mental model is "will return," not "can't here." No design system prescribes a distinct visual tier for this third case; it is conveyed by wording in the sources, and the lock glyph is a convention-aligned but locally untested invention. The single most important rule: **do not grey read-only values to the same faint tone as blocked controls** — that is the current clutter and the readable data the driver wants is exactly what is being dimmed into illegibility.

Making the reason discoverable without permanent clutter follows a two-layer pattern: a **short, always-visible hint** plus a **fuller on-demand explanation**. Because a car screen is touch-only with no hover, the fit is the project's existing `off` string rendered as the always-visible one-line hint, with a tap on the row (or an info affordance) revealing the fuller reason and next step (UX Tigers, Inactive Buttons; Cloudscape, disabled and read-only). Templated wording maps straight onto the catalog's reason constants and makes clutter recede because the eye learns one shape: parked-only → *"Available when parked."*; head-unit-owned (`OWNED`) → *"The head unit controls this."*; no CAN message (`NO_MSG`) → *"Not available on this car."*; mockup (`MOCK`) → *"Preview — not yet connected."* For accessibility, prefer `aria-disabled="true"` with an enforced no-op over native `disabled` so a blocked control stays focusable and its reason is reachable by assistive tech, and render read-only values as readable focusable text (or `aria-readonly`) rather than a native-disabled control that a future TalkBack pass would skip (MDN, aria-readonly; NN/g, Button States). Where many rows are non-actionable, calm the screen within grey-not-hide by grouping by actionability and deferring the unavailable group behind progressive disclosure — a collapsible "Available when parked (8)" summary is especially valuable while driving, cutting clutter exactly when glance time is scarcest, since the collapsed header can carry the shared reason (NN/g, Progressive Disclosure; Cloudscape, disabled and read-only). Caveats to carry: no controlled A/B study compares grey-vs-hide for in-vehicle controls specifically (the automotive sources are manuals and convention), Nielsen is explicit his inactive-button guidance rests on "the balance of arguments and limited evidence, not strong test data," and one team's muted-primary-beats-grey finding is unpublished with "no credible empirical evidence" isolating the effect (UX Tigers, Inactive Buttons).

While driving, still the motion; at night, hold a brightness floor

The governing automotive principle is not "no animation" but "no non-driving dynamic content that draws the eye while moving." NHTSA's voluntary guidelines encode this as **per se lockouts** — displaying text to read, manual text entry, graphical images, video, dynamic maps, and automatically scrolling lists/text are to be disabled while driving — and JAMA states it directly: entertainment content should be avoided and driving operation inoperative

while the vehicle is in motion (NHTSA via UNT archive; JAMA Guideline v3.0). A driving-simulator study pins the specific hazard: an animation **salient by color, brightness, and movement** harms visual attention, and high-distraction animation should be removed from the design from the start (arXiv 1805.02787). For Pulse this cleanly separates the 3D world's motion, which *represents the driving scene* (road, lanes, the car's own motion) and is driving-related information these rules do not target, from decorative transitions, animated empty-state chrome, auto-scrolling music/nav text, and ambient motion, which should be stilled, dimmed, or removed while moving and restored when parked — the literal embodiment of NHTSA's "dim/blank/reposition non-safety dynamic info." WCAG 2.3.3 and `prefers-reduced-motion` back the same move: check `window.matchMedia('(prefers-reduced-motion: reduce)')` and switch non-essential animation to an instant state change or an opacity fade rather than removing feedback (W3C, SC 2.3.3). Rivian is the cautionary twin — an engine-driven 3D HUD praised for looks but criticized for lag and too-many-taps while driving — reinforcing that Pulse's driving mode should stay shallow, immediate, and animation-light, with rich scene motion reserved for parked (InsideEVs, Rivian review).

These driving suppressions ride on a single motion signal — the earlier options report proposes a page-side `driving` flag from `VCU_GearSig` with a speed fallback — which is the same flag the parked-only control grouping and the trial's parked-only gate already want. The glance budget bounding how much a driver can absorb per state change is NHTSA's **≤2 s single glance and ≤12 s total eyes-off-road per task**, with Google's car-UI guidance adding a **76×76 dp minimum touch target, ≥23 dp spacing, 32 dp primary / 24 dp secondary text, and ≥4.5:1 contrast for info-bearing content** (US DOT release; Google, Sizing; Google, Visual principles). A state cue read while moving should therefore be comprehensible in one ~2 s glance, which favors a coarse, high-contrast color/position/icon change on the control itself over text that must be read or motion that must be watched to be understood. ISO 15008's legibility minimums (character height ~20 arc-min recommended, contrast ≈3:1) sit behind a paywall and are secondary/approximate, but the validated finding that **colour difference plays only a minor role in daylight legibility** is firm and useful: luminance contrast, not hue, is the lever for glanceability, so neither theme may rely on color alone to separate text from background (SAE 2011-01-0597; ISO 15008:2017 catalogue).

Night mode is the inverse problem: the screen must not become a glare source. Google's car-UI guidance requires **negative polarity (light on dark) at night because positive polarity can dazzle drivers**, and the recurring failure modes are instructive — a Tesla owner reports light-gray fonts and **weak contrast in both day and night modes**, a direct warning that a minimalist dark theme can quietly fall below legibility, and Rivian shipped fixes for broken auto screen brightness and a display icon that mis-switched day/night, showing that auto day/night is a frequent defect surface (Google, Visual principles; Tesla Motors Club forum; Rivian 2024.07 notes). So the night theme should hold a **calibrated minimum brightness and avoid pure-white large fills** that become glare sources, preserve luminance contrast for text, drive the day/night switch from the car's own ambient/headlight state with **hysteresis** to avoid flicker and mis-trigger, and may use warm/low-blue accents (a comfort/glare lever, not a legibility substitute, since hue barely affects legibility). No authoritative numeric night-luminance floor or ceiling surfaced in accessible sources; ISO 15008 and SAE J1757 hold the measurement framework but the numbers are paywalled.

Empty and offline states round out the polish, and the automotive guidance is sparse but consistent: show an explicit **"unknown / not available" state rather than freezing the last reading or guessing**, keep the slot visible and marked unavailable without reflowing the layout or raising alarm chrome, and reserve alarming visuals for genuine safety conditions (engineered.at, instrument clusters; US11074122B2, graceful UI degradation). This

endorses the repo's existing pattern — a control that cannot send stays greyed with a reason rather than hidden — and extends it to the HUD's "NO RADAR," offline, "MOCKUP," and "Starting the HUD" states: present them calmly, do not use red/urgent styling for the benign "comma offline" or "sensor absent" case, and for the fusion display visibly distinguish **live-but-zero from stale/absent** data (a dimmed or empty gauge for absent, not a held value) so a lost comma or radar source never reads as a frozen real reading. Carry the caveat that none of these empty-state treatments comes from a safety standard — they are HMI-engineering articles and general resilience patterns — and that the NHTSA figures throughout are voluntary 2013 recommendations, the touch-latency numbers are from patents and HCI research rather than any automotive standard, and OEM specifics are limited to Tesla and Rivian (MBUX, Polestar, and the stock Fisker UI did not surface in accessible sources). Euro NCAP's 2026 push toward physical controls for safety functions does not bind an aftermarket center-screen app but is the clearest current signal that deep touchscreen menus for anything safety-adjacent are being graded down (Carwow, Euro NCAP).

A prioritized polish backlog, cheapest wins first

The table orders polish items by value-per-effort, cheapest high-impact wins at the top. "Surface" names the file or module from the app context; effort is S (hours), M (a day or so), L (multi-day); "Needs Android" flags anything beyond the plain ES-module page.

#	POLISH ITEM	WHAT IT CHANGES	SURFACE	EFFORT	PAGE-ONLY?
1	**FPS/debug chip off by default**	`showFps: true → false`; chip no longer overlays every screen	`main.js` (defaults, ~line 33)	S	Page-only
2	Instant press acknowledgement <100 ms	Cheap state-layer press-state (pressed 10%) on every control tap, decoupled from the 3D frame	`carcontrols.js`, `app.css`	S/M	Page-only
3	Build the missing `pending` state	The ~1.5 s pending/confirmed cycle DESIGN.md specifies but code lacks; settle on the car's reply, not the tap	`carcontrols.js` set(), `cancmd.js` resolution	M	Page-only
4	Templated, one-shape reason strings	Rewrite `OWNED` / `NO_MSG` / `MOCK` /parked-only into one consistent sentence each	`carcatalog.js` (`off` constants)	S	Page-only

#	POLISH ITEM	WHAT IT CHANGES	SURFACE	EFFORT	PAGE-ONLY?
5	Split read-only from disabled visually	Read-only rows full-contrast 4.5:1 readable; disabled muted-but-legible; stop greying live values to the blocked tone	`carcontrols.js` state(), `app.css` tokens	M	Page-only
6	Lock glyph + “available when parked” tier	Third visual state for temporarily-unavailable (parked-only/source-off) distinct from permanently blocked	`carcatalog.js`, `carcontrols.js`, `app.css`	S/M	Page-only
7	Trial override snap-back + inline note	Optimistic send → watch car frames ~1.5–2.5 s → visible snap-back to car value + brief inline note when overridden/didn’t-stick	`carcontrols.js` set()/reconcile, `e2e.js`	M	Page-only
8	`aria-disabled` / `aria-readonly` semantics	Keep blocked controls focusable with enforced no-op; read-only values as readable focusable text	`carcontrols.js`	S	Page-only
9	Tap-to-expand fuller reason	Short always-visible hint + tap reveals full reason/next step (touch has no hover)	`carcontrols.js`	S/M	Page-only

#	POLISH ITEM	WHAT IT CHANGES	SURFACE	EFFORT	PAGE-ONLY?
10	Still non-driving motion while moving	Suppress decorative transitions, animated empty-state chrome, auto-scroll; honor `prefers-reduced-motion`; keep scene motion	`scene.js`, `main.js` (driving flag), `app.css`	M	Page-only
11	Collapse parked-only group while driving	One “Available when parked (n)” summary row in motion, expand when parked	`carcontrols.js` + driving flag	M	Page-only
12	Calm empty/offline states	“NO RADAR”/offline/MOCKUP without alarm chrome; distinguish live-zero from stale/absent; no frozen readings	scene/HUD empty-state modules	S/M	Page-only
13	Snackbar discipline	Reversible actions → snackbar w/ Undo at 8–10 s; errors inline near the control; toast supplementary, never blocking	`carcontrols.js` toast	S/M	Page-only
14	M3 motion timing scale	Press/state 50–200 ms, panels 250–400 ms, ripple ~75 ms fade; standard easing	`app.css`	S	Page-only
15	Night brightness floor + contrast audit	No pure-white large fills at night; verify luminance contrast ≥4.5:1 in both themes (don’t rely on hue)	`app.css` theme tokens	M	Page-only

#	POLISH ITEM	WHAT IT CHANGES	SURFACE	EFFORT	PAGE-ONLY?
16	Haptic/audio tap confirm	Non-visual acknowledgement so the driver confirms without a glance	`carcontrols.js` → `window.WebHudApp`	M	Needs Android
17	Hysteresis auto day/night from car	Drive the theme switch from car ambient/headlight state with hysteresis to stop flicker/mis-trigger	worker signal plumbing + `app.css`	M	Needs signal/worker

Development plan

The backlog above is ordered by value-per-effort; this plan sequences it into shippable phases. The organizing facts: almost every item is **page-only**, sends **no new CAN frames**, and costs **no frame time**, so it ships incrementally behind whichever layout the earlier report settles on. The one cross-cutting dependency is the page-side `driving` **flag** (from `VCU_GearSig` with a speed fallback, as the options report proposes) — it is the spine that gates items 7, 10 and 11, so it is built early. Visual changes are verified in the **browser harness** (headless-Chrome screenshots at Pulse size); the two behaviours the research flags as convention-not-proof — the override snap-back and the empty/offline states — get an **on-car check** before they are trusted.

PHASE	GOAL	ITEMS	DEPENDS ON	EFFORT	VERIFY
0 — Quick wins	Immediate “looks finished” lift, zero behaviour risk	1 (FPS chip off), 4 (reason strings), 14 (motion tokens)	—	~½ day	Harness screenshots
1 — Driving flag	The foundation the rest leans on	page-side `driving` flag	—	~½ day	Unit test + harness (synthetic gear/speed)
2 — Feedback loop	Let the car, not the tap, be the source of truth	2 (press ack), 3 (pending→confirmed), 7 (override snap-back), 13 (snackbar discipline)	3 precedes 7	~2–3 days	Harness for visuals; **on-car** for the override
3 — Control clarity	Split read-only / disabled / parked-only	5 (read-only vs disabled), 6 (parked-only lock tier), 8 (aria semantics), 9 (tap-to-expand reason)	4; `state()` refactor	~2–3 days	Harness per state; accessibility pass

PHASE	GOAL	ITEMS	DEPENDS ON	EFFORT	VERIFY
4 — Driving-aware polish	Calm the screen while moving	10 (still non-driving motion), 11 (collapse parked-only group), 12 (calm empty/offline)	Phase 1 flag	~2 days	Harness parked vs driving; **on-car** on a drive
5 — Theme & hardware	Night legibility and non-visual confirm	15 (night floor + contrast), 16 (haptic/audio), 17 (auto day/night hysteresis)	16/17 need Android / a worker signal	~2 days	Harness dark theme; **on-car** for 16/17

Sequencing rationale. Phases 0–1 are shippable in a day and de-risk nothing, so they go first. Phase 2 is the heart of the "state honesty" thesis and is self-contained page-side. Phase 3 depends only on the Phase 0 reason strings plus a `state()` refactor, so it can run in parallel with Phase 2 if there are two hands. Phase 4 cannot start until the Phase 1 flag exists. Phase 5 is last because two of its three items are the only ones that reach past the page into the Android app and the worker. Rough total: **~10–12 working days**, front-loaded so the visible payoff lands in the first.

What stays out of scope. This plan does not touch the layout decision (that is the earlier options report), does not add CAN messages, and does not change what the app sends — it changes only how the app *tells the driver what happened*. Each phase can ship on its own behind the existing settings, so none of it blocks a release.

Conclusion

The HUD's unfinished feel is not a shortage of features but a shortage of state honesty: the app sends and reads correctly, yet it does not yet tell the driver clearly what happened to a tap, what a greyed row means, or when it is showing stale data. The evidence resolves this into one organizing idea — let the car, not the touch, be the source of truth — which turns out to answer all three question areas at once. Feedback becomes "acknowledge instantly, confirm from the car's reply"; the trial override stops being a silent failure and becomes a visible reconcile-to-canonical snap-back; and the control catalog's flat grey splits into read-only data the driver should read, blocked controls with discoverable reasons, and parked-only controls that announce they will return. Almost every item is page-only, costs no frame time, and sends no new CAN frames, which is what makes this layer cheap to ship incrementally behind whichever layout the earlier report settles on.

The sharpest implication is that the driving/parked boundary is the spine of the whole polish layer, not just a motion concern: the same `driving` flag gates the motion suppression, the parked-only group collapse, and the trial's existing parked-only lock, so building that flag well pays for items 7, 10, and 11 together. Two honesty markers should travel with any implementation — the automotive numbers are voluntary and partly paywalled (NHTSA 2013 recommendations, ISO 15008 values unverified), and optimistic UI is convention rather than a validated finding — so the override and empty-state behaviors in particular deserve a quick on-car check rather than being trusted from the literature alone.